

AUTOMATUL DE COMANDĂ ȘI CONTROL CU MEMORIE ROM

Conținut laborator:

- 1. Obiectivul lucrării.**
- 2. Noțiuni teoretice:**
- 3. Cerințe laborator.**
- 4. Temă.**

Obiectivul lucrării:

În acest laborator este tratată o aplicație a memoriilor ROM și anume automatul programabil CROM. La baza acestuia stă un circuit de memorie ROM și un numărător de adrese cu incrementare. În plus, pot exista și circuite demultiplexoare pentru extinderea magistralei de comenzi dar și multiplexoare de teste. Automatul CROM stă la baza unității de execuție care există într-un procesor. Cunoașterea acestuia este un prim pas în înțelegerea mecanismului de execuție a instrucțiunilor și de efectuare a testelor și a salturilor în memoria de programe.

Utilitatea laboratorului constă în primul rând în suportul tehnic pe care îl aduce pentru înțelegerea mecanismelor care au loc într-un procesor. Pe de altă parte, automatele CROM sunt utilizate în proiectarea cu circuite reconfigurabile, la construirea unor nuclee programabile, capabile să execute seturi secvențiale de comenzi.

Noțiuni teoretice:

A. Memoriile ROM în aplicații

În lucrul cu memoriile ROM se disting trei etape distincte: programarea memoriei, ștergerea memoriei și citirea memoriei. Într-o aplicație o memorie ROM este văzută ca un circuit de memorare care permite numai citirea acestuia. Însă, pentru a putea folosi o memorie ROM ea trebuie inițial programată, adică scrisă cu secvența de date dorită. Spre deosebire de clasa memoriilor RAM scrierea (programarea) memoriilor ROM este o etapă distinctă de cea de funcționare normală.

Regimurile de lucru sunt prezentate în figura 1.

Pentru reprogramarea memoriilor din clasa ROM este necesară efectuarea unei operații suplimentare care presupune ștergerea vechiului conținut. Ștergerea reprezintă de asemenea un regim de lucru special de funcționare.

a. Regimul de programare (scriere) al memoriei

Programarea memoriilor ROM este o etapă de lucru distinctă în lucrul cu memoriile ROM. Programarea se poate efectua numai la o locație (sau bloc de locații) goală din memorie. O locație se consideră goală dacă toți biții din ea sunt în 1 logic (de ex. o locație pe un octet goală este FFh).

În funcție de tipul memoriei ROM aceasta presupune următoarele:

- aplicarea unor tensiuni suplimentare, de regulă de valori mai mari decât cele de operare în regim de lucru normal;
- aplicarea unor semnale de validare a programării la aceleași valori de tensiune cu regimul normal de operare;
- aplicarea unor algoritmi care demarează programarea memoriei, fără a fi necesar aplicarea unor tensiuni sau semnale de comandă externe suplimentare.

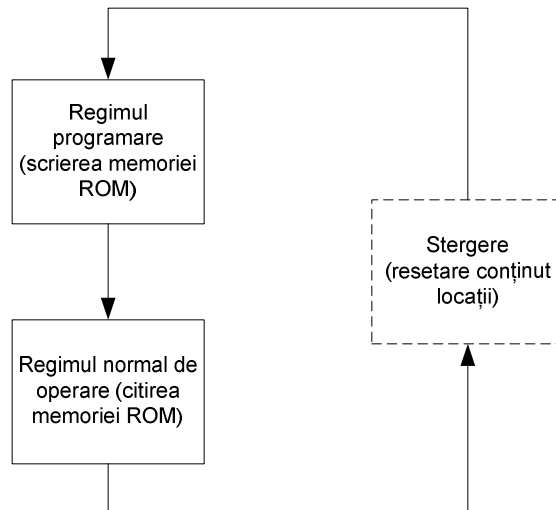


Fig.1. Lucrul cu memoria ROM

La prima grupă memoria este organizată ca în figura de mai jos:

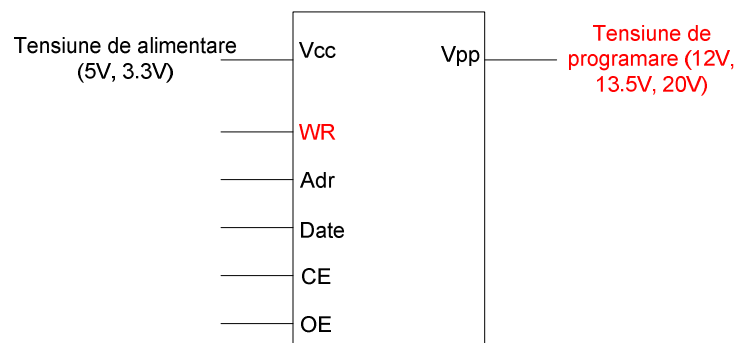


Fig.2. Memorii cu programarea prin aplicarea unor tensiuni de programare

Programarea memoriei presupune existența unor tensiuni de programare, mai mari decât tensiunea de lucru normal (V_{cc}). Pentru aplicarea acestor tensiuni sunt deci necesare surse de tensiune separate.

Activarea programării se face prin setarea semnalului WR. Din această categorie fac parte circuitele ROM de tip OTP (one time programming).

La cea de-a doua grupă memoria este organizată astfel:

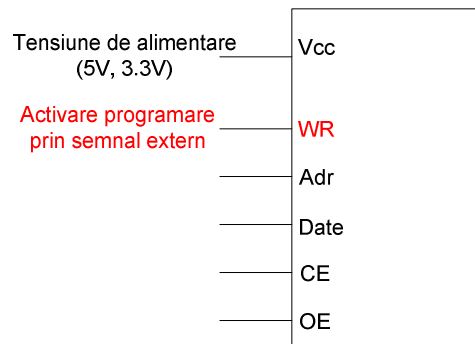


Fig.3. Memorii cu programarea prin aplicarea unui semnal extern

Aici programarea se face prin aplicarea, pe un pin al memoriei, al unui semnal de comandă (WR), la tensiunea normală de operare a memoriei. În regimul normal de lucru acest semnal trebuie setat pe un nivel inactiv. Din această grupă se fac parte memoriile EEPROM.

Cea de-a treia categorie este în felul următor:

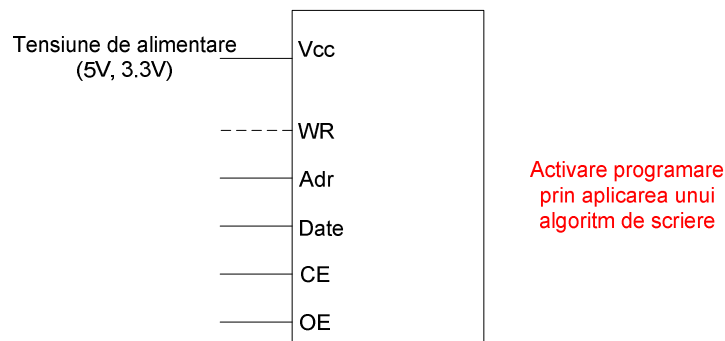


Fig. 4. Programarea memoriei cu algoritm de scriere

Această a treia categorie presupune, pentru activarea programării, plasarea unui algoritm de scriere, adică o anumită secvență de adrese și date care va declanșa etapa de programare. La aceste circuite poate fi prezent, opțional, un semnal de activare a scrierii de tipul WR, extern.

Din această a treia categorie fac parte memoriile FLASH.

b. Regimul de ștergere (scriere) al memoriei

Ștergerea este o etapă care presupune trecerea unui bit din 0 logic în 1 logic, cu alte cuvinte golirea unei locații de memorie. În funcție de capacitatea de ștergere se întâlnesc următoarele tipuri de memorii ROM:

- memorii ROM la care ștergerea nu este posibilă: memorii OTP (one time programming) sau memorii EPROM;
- memorii la care ștergerea se efectuează prin plasarea memoriei într-un câmp de radiații ultra-violete – memorii EPROM cu ștergere prin ultraviolete;

- memorii la care ștergerea se efectuează prin aplicarea unor semnale electrice pe pini speciali ai memoriei – memorii EEPROM (memorii EPROM cu ștergere electrică);
- memorii la care ștergerea se efectuează prin aplicarea pe adrese și date a unor algoritmi speciali de ștergere (secvențe de adrese și date): memorii FLASH.

c. Regimul normal de lucru al memoriei ROM (citire)

Este regimul prin care se operează cel mai mult cu memoria ROM. Ea este văzută practic ca un circuit care permite numai citirea locațiilor din memorie. Modul de lucru aici este unul similar cu cel efectuat la memoriile din clasa RAM la operația de citire. Toate celelalte semnale utilizate pentru programare și ștergere sunt aduse la nivele inactive.

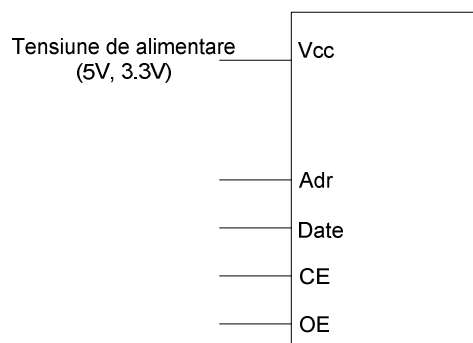


Fig.5. Memoria ROM utilizată în aplicații

Citirea conținutului se face prin aplicarea adresei și a semnalelor de comandă (CE și OE) toate acestea la tensiunea de alimentare normală.

Programarea și ștergerea memoriilor se efectuează, la toate tipurile de ROM unde sunt posibile, de un număr limitat de ori precizat în datele de catalog (variază între 100 – 100000 în funcție de tehnologie).

În acest laborator se va lucra cu o memorie ROM cu organizarea 256x8 (prezentată în laboratorul anterior), cu următoarele caracteristici:

Circuitul rom256x8 este o memorie ROM de 256 de octeți.

Memoria rom256x8:

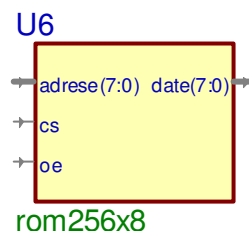


Fig.6. Memoria ROM 256x8

Are 256 locații pe 8 biți fiecare.

Intrări:

Adrese(7..0) - intrare adrese – 8 biți;

CS – intrare activă 0 logic. Activează întreg circuitul.

OE – intrare activă 0 logic. Activează liniile de date.

Ieșiri:

Date(7..0) – ieșire date - 8 biți.

Așa cum se poate observa, memoria ROM256x8 este proiectată ca o memorie accesibilă numai operației de citire. Fiind vorba de o descriere în VHDL pentru implementarea pe machetele din laborator, nu sunt prezente semnalele de programare, ca la memoriile din clasa ROM reale. Etapele de programare și ștergere a acestei memorii constau în modificarea codului VHDL în care este descrisă memoria și conținutul său.

Listingul complet al programului VHDL care descrie comportamentul memoriei a fost prezentat în laboratorul anterior (laboratorul 3).

B. Automatele programabile cu memorii ROM

Una din utilizările cel mai des întâlnite ale memoriei ROM este stocarea unor secvențe de instrucțiuni care apoi să fie „rulate” cu un montaj de genul celui din figura de mai jos:

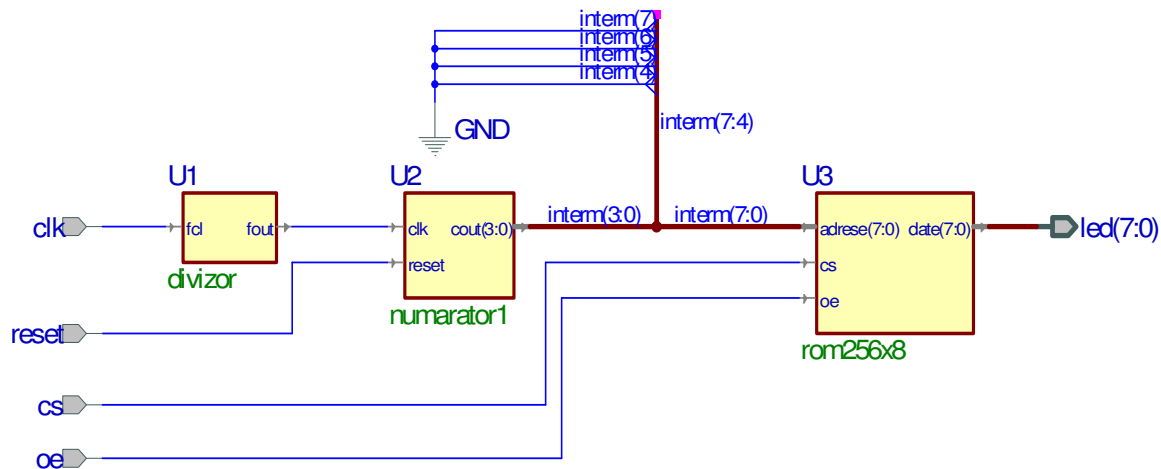


Fig. 8. Executarea automată a instrucțiunilor dintr-un ROM prin parcurgerea adreselor

Schema din figura 8 se bazează pe un circuit de memorie ROM și pe un circuit numărător cu incrementare. Așa cum se poate observa, numărătorul are ieșirile conectate la intrările de adrese ale memoriei ROM. Astfel, în ritmul semnalului de ceas care îl primește pe intrarea clk, acesta va incrementa valoarea adresei memoriei ROM având drept efect parcurgerea liniară a secvenței de comenzi care se află stocată în memoria ROM la locații succesive.

O primă condiție ca sistemul din figura 8 să funcționeze este ca intrările de comandă CE și OE să fie activate. La resetare, ieșirile numărătorului sunt puse în 0 logic. Deci și adresa pe care acesta o va accesa este 00h. Prin generarea unui front activ de ceas (front crescător) ieșirea numărătorului se va incrementa (se va aduna 1) prin urmare se va accesa următoarea adresă.

Deci, prin parcurgerea liniară a adreselor memoriei se pot genera, în ordine, comenzi în conformitate cu instrucțiunile din memoria ROM.

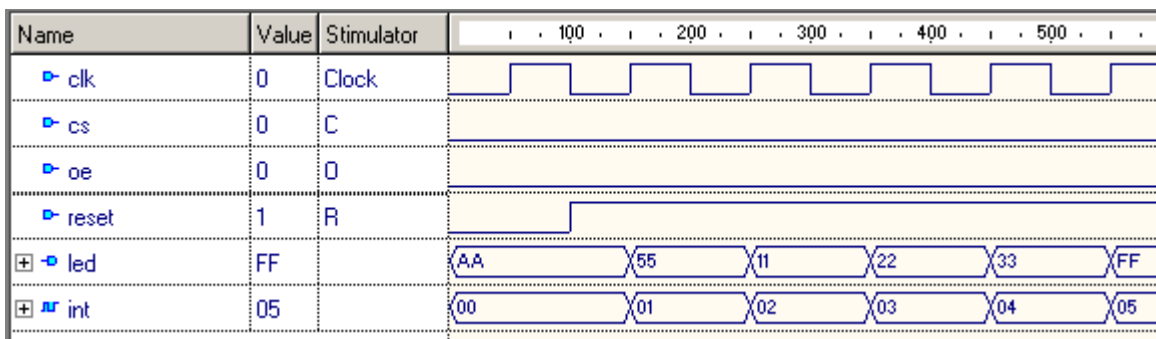


Fig.9. Răspunsul circuitului cu memorie și numărător

Așa cum se poate observa în figura 9, atunci când CS și OE sunt active la ieșirea memoriei ROM (led) se succed secvențe de instrucțiuni în ritmul unui semnal de ceas. Acestea pot fi folosite pentru comanda unor circuite externe.

Executarea, instrucțiune cu instrucțiune, a secvențelor de program este o caracteristică tipică sistemelor de calcul. Unitatea de execuție dintr-un procesor are la bază acest ansamblu memorie de program - circuit numărător de adrese.

Structura hardware prezentată în figura 8 poate executa secvențe de instrucțiuni, cum ar fi de tipul celei de mai jos (exemplu în limbajul C):

```
a=10;
b=20;
c=30;
a++;
b++;
c++;
s=a+b;
s+=c;
```

Acestea se numesc secvențe de program liniare. În proporție de 80% orice program (secvență de instrucțiuni) este constituit din secvențe liniare.

Totuși, pe alocuri, programul conține și instrucțiuni decizionale și de salt (branch, jump) de tipul celor prezentate mai jos (exemple în limbajul C):

```
for(i=0;i<100;i++)
{
}

if(a==b)
{
}
```

Prima execută un test al variabilei i și intră în buclă dacă condiția este falsă iar cea de-a doua testează egalitatea între a și b și sare peste blocul de instrucțiuni următoare dacă condiția este falsă.

La baza acestor instrucțiuni stau testele și salturile în memoria de program. Deci, alături de execuția liniară, prin parcurgerea succesivă a locațiilor din memorie, un program mai efectuează și salturi la adrese.

Pentru implementarea unui automat care poate executa liniar instrucțiuni dar poate și efectua salturi se folosește o schemă de tipul celei din figura 10.

Circuitul „numarator2” este un numărător pe 4 biți cu reîncărcare. Dacă pe pinul „load” (activ 0 logic) se aplică un semnal 0 logic atunci datele de la intrarea „cin” se vor încărca în numărător și se vor regăsi pe ieșirea „cout” a acestuia. Pe intrările circuitului multiplexor 4:1 se vor afla semnalele care comandă intrarea „load” a numărătorului. Dacă este selectată intrarea 0 atunci load va fi permanent în 1 logic deci orice salt în memoria de program este dezactivat. Aceasta este poziția pentru execuția liniară a secvențelor de instrucțiuni, adică parcurgerea locație cu locație. Dacă sunt selectate intrările 1 sau 2 de la multiplexor atunci comanda „load” la numărător este dată de starea pinilor de testare „T1” sau „T2”. Dacă valoarea care se află pe aceste intrări este 0 logic atunci se efectuează un salt în memoria de program, altfel se execută instrucțiunea aflată la adresa următoare. Putem vorbi despre salturi condiționate de valoarea intrărilor de test T1 sau T2. Dacă este selectată intrarea 3 atunci comanda „load” va fi permanent 0 logic deci numărătorul va lucra în modul salt. Vorbim de un salt necondiționat. Circuitul demultiplexor 1:4 asigură 4 funcții de comandă distincte.

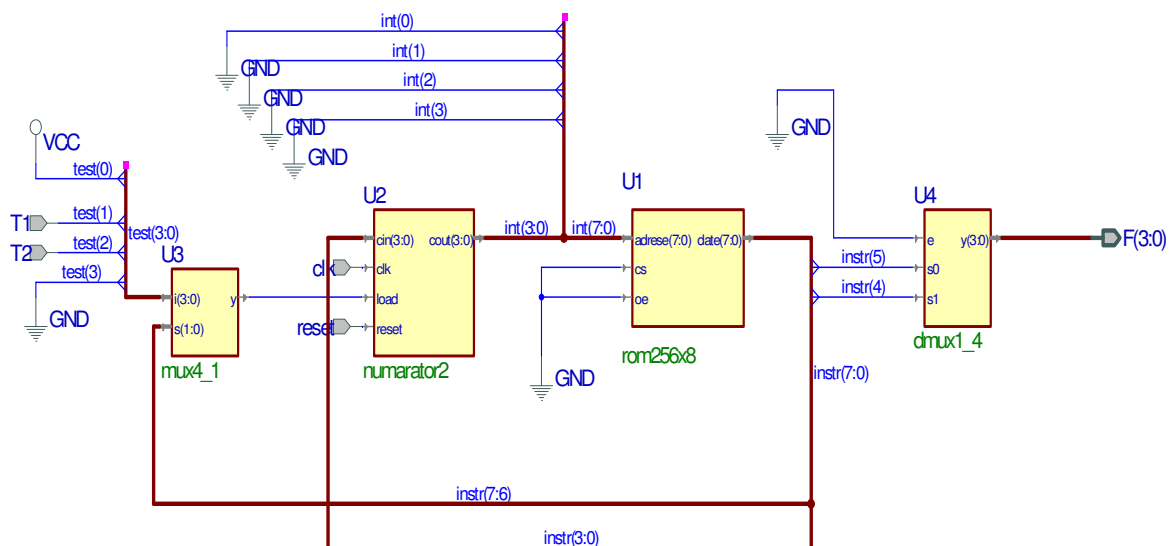


Fig.10. Automatul de comandă și control cu 4 funcții, 2 teste, 2 salturi condiționate și un salt necondiționat

Instrucțiunea care se află în memoria ROM are următoarea configurație:

Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
Selecție test		Selecție funcție		Adresă de salt pentru test			

Pornind de la structura instrucțiunii se definește următorul set de instrucțiuni:

Cod binar	Cod hexa	Simbol	Explicație
0000xxxx	00h	F0	Execută funcția F0
0001xxxx	10h	F1	Execută funcția F1
0010xxxx	20h	F2	Execută funcția F2
0011xxxx	30h	F3	Execută funcția F3
01xxaaaa	4(A)	T1? : A	Salt la adresa A (aaaa) dacă T1 este 0
10xxaaaa	8(A)	T2? : A	Salt la adresa A (aaaa) dacă T2 este 0
11xxaaaa	C(A)	:A	Salt necondiționat la adresa A (aaaa)

x: dont care

aaaa, A : adresa de salt pe 4 biți

Cerințe laborator:

Probleme propuse

- (implementare macheta)** Să se descrie în VHDL o memorie ROM având scrisă următoarea secvență:

(00) = 13h (la adresa 00 scrie valoarea 13h)

(01) = 23h

(02) = 20h

(03) = 02h

(04) = 78h.

Să se proiecteze apoi o structură care să permită citirea și afișarea locațiilor din memorie pe primii doi digiți de la macheta (ca în figura 11).

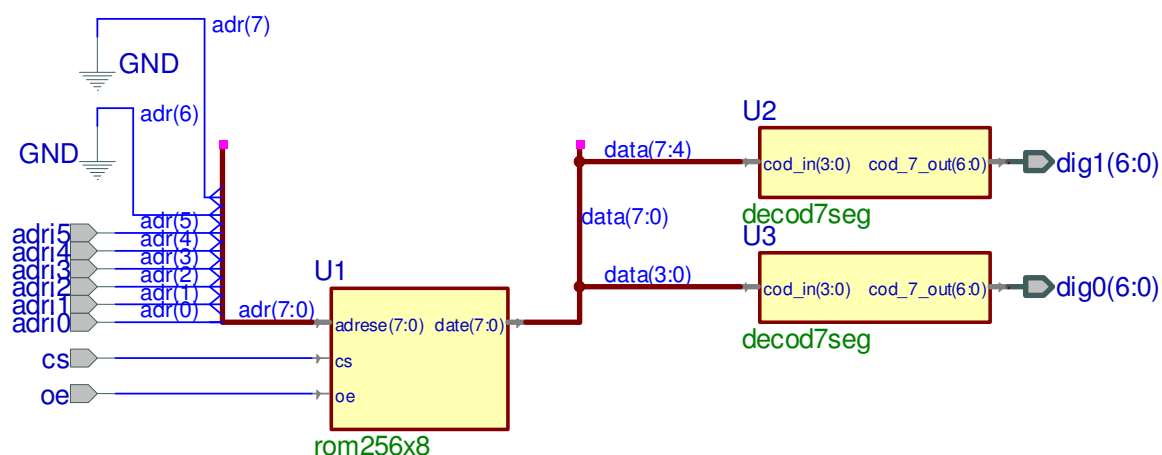


Fig.11. Automatul de comandă și control cu 4 funcții, 2 teste, 2 salturi condiționate și un salt necondiționat

Să se realizeze o secvență care va citi memoria ROM și se vor constata datele programate. (se vor genera din comutatoare semnalele corespunzătoare pentru pinii de adresă, OE și CS).

Rezolvare

- Se va crea un proiect în ActiveHDL. Se va crea un fișier rom256x8.vhd utilizând Wizard. Se va scrie codul VHDL pentru memoria ROM256x8, cu structura prezentată în figura 6, inserând conținutul cerut.

Fișierul rom256x8.vhd va conține următorul program:

```
library IEEE;
use IEEE.STD_LOGIC_1164.all;

entity rom256x8 is
    port(
        cs : in STD_LOGIC;
        oe : in STD_LOGIC;
        adrese : in STD_LOGIC_VECTOR(7 downto 0);
        date : out STD_LOGIC_VECTOR(7 downto 0)
    );
end rom256x8;

architecture rom256x8_a of rom256x8 is
    signal data_in : std_logic_vector(7 downto 0);
    begin

        with adrese select
            data_in <=

                --aici urmeaza continutul memoriei ROM
                --data      adresa
                X"13" when X"00",
                X"23" when X"01",
                X"20" when X"02",
                X"02" when X"03",
                X"78" when X"04",
                X"FF" when others;

        process(cs,oe,data_in)
        begin
            if cs='0' and oe = '0' then
                date <= data_in;
            else
                date <= "ZZZZZZZZ";
            end if;
        end process;
    end rom256x8_a;
```

- Se va crea un fișier vhd, denumit decod7seg.vhd în care se va introduce codul pentru decodorul 7 segmente. Structura acestuia este prezentată în figura de mai jos:

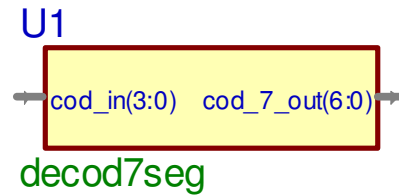


Fig.12. Decodorul 7 segmente

Codul VHDL care trebuie să se regăsească în fișier este prezentat mai jos:

```
library IEEE;
use IEEE.STD_LOGIC_1164.all;

entity decod7seg is
    port(
        cod_in : in std_logic_vector(3 downto 0);
        cod_7_out : out std_logic_vector(6 downto 0)
    );
end decod7seg;

architecture decod7seg_a of decod7seg is
begin

    process(cod_in)
    begin
        case cod_in is
            when "0000" => cod_7_out <= "0111111";
            when "0001" => cod_7_out <= "0000110";
            when "0010" => cod_7_out <= "1011011";
            when "0011" => cod_7_out <= "1001111";
            when "0100" => cod_7_out <= "1100110";
            when "0101" => cod_7_out <= "1101101";
            when "0110" => cod_7_out <= "1111101";
            when "0111" => cod_7_out <= "0000111";
            when "1000" => cod_7_out <= "1111111";
            when "1001" => cod_7_out <= "1101111";
            when "1010" => cod_7_out <= "1110111";
            when "1011" => cod_7_out <= "1111100";
            when "1100" => cod_7_out <= "0111001";
            when "1101" => cod_7_out <= "1011110";
            when "1110" => cod_7_out <= "1111001";
            when "1111" => cod_7_out <= "1110001";
            when others => cod_7_out <= "0000000";
        end case;
    end process;
end;
```

```

end case;
end process;
end decod7seg_a;

```

- Pentru a putea realiza citirea și afișarea rezultatului se vor folosi două decodoare BCD-7 segmente. Se va crea un fișier schematics (block diagram). Se vor utiliza cele două componente descrise anterior (memoria ROM și decodorul 7 segmente). Conexiunea se componentelor se realizează ca în figura 11.

Reamintim că pentru denumirea magistralelor sau liniilor se dă dublu click pe magistrala respectiv linia care se dorește redenumită și se tastează denumirea. Pentru magistrale se specifică dimensiunea iar pentru fire conectate la magistrală se folosesc aceleași denumiri cu ale magistralei iar între paranteze se specifică numărul firului în magistrală (vezi laboratoarele anterioare).

Pentru implementare se vor urmări pașii din anexă. Distribuirea pinilor este specificată în tabelul de mai jos:

Porturi	Macheta	Specificatii
adri0	sw_dip0	Switch 1 – ON = 0, OFF = 1
adri1	sw_dip1	Switch 2 – ON = 0, OFF = 1
adri2	sw_dip2	Switch 3 – ON = 0, OFF = 1
adri3	sw_dip3	Switch 4 – ON = 0, OFF = 1
adri4	sw_dip4	Switch 5 – ON = 0, OFF = 1
adri5	sw_dip5	Switch 6 – ON = 0, OFF = 1
cs	sw_dip6	Switch 7 – ON = 0, OFF = 1
oe	sw_dip7	Switch 8 – ON = 0, OFF = 1
dig1(0)	dig1_seg0	digit 1 segment 0
dig1(1)	dig1_seg1	digit 1 segment 1
dig1(2)	dig1_seg2	digit 1 segment 2
dig1(3)	dig1_seg3	digit 1 segment 3
dig1(4)	dig1_seg4	digit 1 segment 4
dig1(5)	dig1_seg5	digit 1 segment 5
dig1(6)	dig1_seg6	digit 1 segment 6
dig0(0)	dig0_seg0	digit 0 segment 0
dig0(1)	dig0_seg1	digit 0 segment 1
dig0(2)	dig0_seg2	digit 0 segment 2
dig0(3)	dig0_seg3	digit 0 segment 3
dig0(4)	dig0_seg4	digit 0 segment 4
dig0(5)	dig0_seg5	digit 0 segment 5
dig0(6)	dig0_seg6	digit 0 segment 6

2. Să se programeze o memorie ROM cu secvența următoare:

```

(00) = 81h
(01) = 42h

```

(02) = 24h
 (03) = 18h
 (04) = 24h
 (05) = 42h
 (06) = 81h
 (07) = c3h
 (08) = 66h
 (09) = 3ch
 (10) = 66h
 (11) = c3h
 (12) = e7h
 (13) = 7eh
 (14) = ffh
 (15) = 00h
 (16) = ffh

Să se realizeze apoi circuitul din figura 8 pentru executarea automată a instrucțiunilor, pas cu pas.

1. Se va crea un proiect. Se va crea un fișier rom256x8.vhd în care se va descrie memoria ROM cu secvența cerută (se urmăresc pașii de la punctul anterior);
2. Se va crea un fișier vhdl pentru divizorul de ceas (divizor.vhd). Acesta va avea următoarea structură.

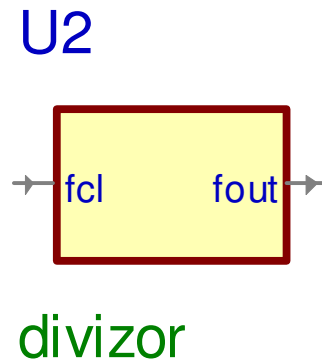


Fig.13. Divizorul de frecvență (1:50000000)

Codul VHDL pentru divizorul de frecvență este următorul:

```

library IEEE;
use IEEE.STD_LOGIC_1164.all;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity divizor is
  Port(
    fcl : in std_logic;
    fout : out std_logic
  )

```

```

);
end divizor;
architecture divizor_a of divizor is

signal decnt : std_logic_vector(25 downto 0);
signal fout_int : std_logic;

begin

    deb : process (fcl)
    begin
        if fcl'event and fcl = '1' then
            if decnt < 500000000 then --constata divizare
                decnt <= decnt + '1';
                fout_int <= '0';
            else
                fout_int <= '1';
                decnt <= "00000000000000000000000000000000";
            end if;
        end if;
    end process;
    fout <= fout_int;
end architecture;

```

3. Se va descrie un numărător pe 4 biți în fișierul numarator1.vhd. Structura acestuia este următoarea:

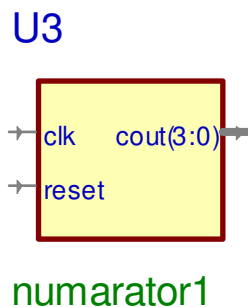


Fig.14. Numărătorul pe 4 biți cu incrementare

Codul VHDL este următorul:

```

library IEEE;
use IEEE.STD_LOGIC_1164.all;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

```

```

entity numarator1 is
    Port(

```

```

        clk : in std_logic;
        reset : in std_logic;
        cout : out std_logic_vector(3 downto 0)
            );
end numarator1;

architecture numarator1_a of numarator1 is

begin

    deb : process (clk,reset)
        variable      decnt : std_logic_vector(3 downto 0);
    begin
        if reset = '0' then
            decnt := "0000";
        elsif clk'event and clk = '1' then
            decnt := decnt + 1;
        end if;
        cout <= decnt;
    end process;

end architecture;

```

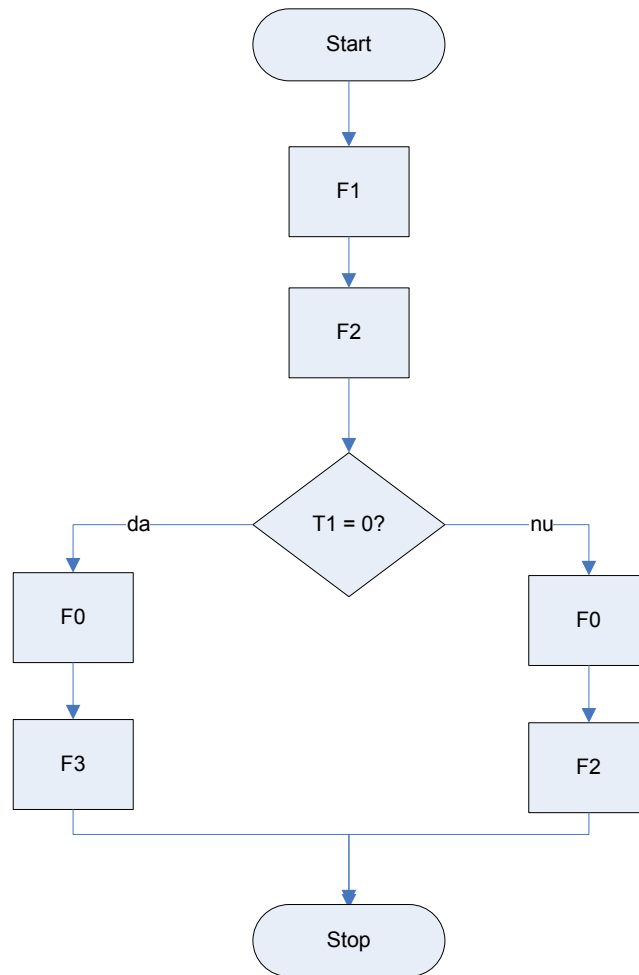
4. Se va construi schema din figura 8. Se va compila și apoi se va implementa.

Pentru implementare avem următoarele distribuiri ale pinilor:

Pin logic	Pin machetă
clk	Fclk
Reset	Sw_user0
Cs	Sw_dip0
Oe	Sw_dip1
Led7..0	Led7..0

Temă

3. Să se realizeze un automat de comandă și control cu 4 funcții și 2 teste și să se implementeze pe el următoarea organigramă:



1. Se construiește un nou proiect. Se va descrie în VHDL o memorie ROM256x8 care are următorul conținut.

Adresa	Operație	Binar	Descriere
00	F1	00010000	Execută funcția F1
01	F2	00100000	Execută funcția F2
02	T1? :06	01000110	Testează T1 dacă e 0 salt la adresa 06h
03	F0	00000000	Execută funcția F0
04	F2	00100000	Execută funcția F2
05	:08	11001000	Salt necondiționat la adresa 08h
06	F0	00000000	Execută funcția F0
07	F3	00110000	Execută funcția F3
08	:00	11000000	Salt la adresa 00h

Se va compila. Se va genera fișierul rom256x8.vhd.

2. Se vor crea fișiere vhd pentru următoarele componente:

Numărătorul 4 biți cu reîncărcare:

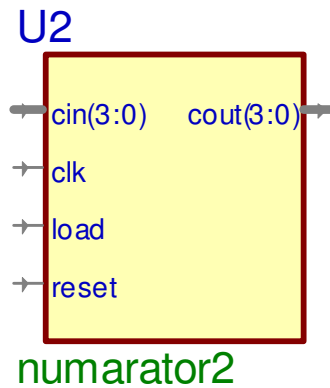


Fig.15. Componenta numărător 4 biți cu reîncărcare

Codul VHDL este următorul:

```
library IEEE;
use IEEE.STD_LOGIC_1164.all;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity numarator2 is
    Port(
        clk : in std_logic;
        reset : in std_logic;
        load : in std_logic;
        cin : in std_logic_vector(3 downto 0);
        cout : out std_logic_vector(3 downto 0)
    );
end numarator2;

architecture numarator2_a of numarator2 is
begin
    deb : process (clk,reset)
        variable decnt : std_logic_vector(3 downto 0);
    begin
        if reset = '0' then
            decnt := "0000";
        elsif clk'event and clk = '1' then
            if load = '1' then
                decnt := decnt + 1;
            else
                decnt := cin;
            end if;
        end if;
    end process;
end numarator2_a;
```



```

        end if;
        cout <= decnt;
    end process;

```

Multiplexor 4:1:

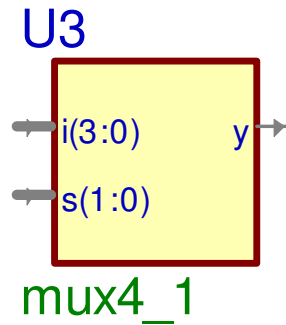


Fig.16. Componenta MUX4:1

Codul VHDL pentru MUX4:1:

```

library IEEE;
use IEEE.STD_LOGIC_1164.all;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity mux4_1 is
    port(
        s : in STD_LOGIC_vector(1 downto 0);
        i : in std_logic_vector(3 downto 0);
        y : out std_logic
    );
end mux4_1;

architecture mux4_1_a of mux4_1 is
begin

    process(s)
    begin
        case s is
            when "00" => y <= i(0);
            when "01" => y <= i(1);
            when "10" => y <= i(2);
            when "11" => y <= i(3);
            when others => y <= 'Z';
        end case;
    end process;

```

end mux4_1_a;

Componenta demultiplexor 1:4

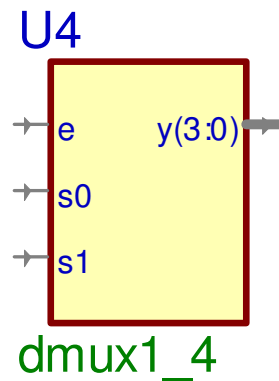


Fig.17. Componenta DMUX1:4

Codul VHDL este:

```
library IEEE;  
use IEEE.STD_LOGIC_1164.all;
```

```
entity dmux1_4 is  
    port(  
        e : in STD_LOGIC;  
        s1 : in STD_LOGIC;  
        s0 : in STD_LOGIC;  
        y : out STD_LOGIC_VECTOR(3 downto 0)  
    );  
end dmux1_4;
```

```
architecture dmux1_4_a of dmux1_4 is  
    signal sint : std_logic_vector(1 downto 0);  
begin
```

```
    sint(0) <= s0;  
    sint(1) <= s1;  
    process(e, sint)  
    begin  
        if e = '0' then  
            case sint is  
                when "00" => y <= "1110";  
                when "01" => y <= "1101";  
                when "10" => y <= "1011";  
                when "11" => y <= "0111";  
                when others => y <= "1111";  
            end case;  
        else
```

```

        y <= "1111";
    end if;
end process;
end dmux1_4_a;

```

4. Se va crea schema din figura 10. Se va compila. Se va implementa

La implementare avem:

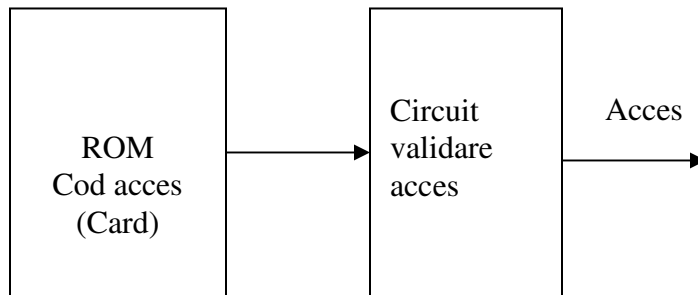
Pin logic	Pin machetă
Clk	Sw_user1
Reset	Sw_user0
T1	Sw_dip0
T2	Sw_dip1
F3..0	Led3..0

4.Pinii date(2), date(3), date(4) și date (5) ale unui montaj de tipul celui din figura 8 sunt conectați cu anodul unor LED-uri (ROSU, GALBEN, ALBASTRU, ALB). Să se programeze memoria și să se genereze apoi semnalele care vor produce următoarele secvențe:

- ROSU, ROSU-GALBEN, ROSU-GALBEN-ALBASTRU, ALB;
- ROSU, GALBEN, ALBASTRU, ALB;
- ROSU-ALBASTRU, GALBEN-ALB.

LED-urile sunt active (aprinse), dacă pe anod se află „1”.

5. Să se realizeze următoarea schemă de circuit control acces prin card:



Ieșirea „Acces” trebuie să fie „1” pentru ca accesul să fie permis.

Codurile de acces valide sunt următoarele: xx1x0010 cu x: 0 sau 1.

Se cer:

- a) Să se scrie în memorie 5 coduri distincte de acces la adresele 00, 45, 56, 67 și 78;

- b) Să se realizeze circuitul de validare acces și să se conecteze memoria realizată;
- c) Să se genereze secvența care citește de la adresele respective. Se va verifica dacă ieșirea „Acces” se setează corespunzător.

Criterii de evaluare:

1	1P
2	2P
3	2P
4	2P
5	3P
6	3P